

Presento la implementación del núcleo funcional y la infraestructura inicial para el videojuego **"El Último Bastión"**, diseñada bajo estrictos criterios de arquitectura limpia, modularidad y desacoplamiento mediante el patrón de Máquina de Estados Finitos (FSM).

1. Archivos Creados

A continuación se detalla la estructura física del proyecto con los archivos necesarios para inicializar el motor base:

```
Plaintext
el_ultimo_bastion/
├── config/
│   └── settings.json
├── src/
│   ├── core/
│   │   ├── __init__.py
│   │   ├── game_manager.py
│   │   └── state_manager.py
│   └── __init__.py
├── main.py
└── requirements.txt
```

2. Código Completo

requirements.txt

```
Plaintext
pygame==2.6.1
```

config/settings.json

JSON

```
{
  "screen": {
    "width": 1280,
    "height": 720,
    "title": "El Ultimo Bastion - Core Engine",
    "fps": 60
  }
}
```

src/core/state_manager.py

Python

import pygame

class State:

"""

Clase abstracta pura (Interfaz) para definir el comportamiento
de los diferentes estados del videojuego.

"""

def __init__(self, state_manager):

self.manager = state_manager

def handle_events(self, events):

"""Procesa entradas de teclado, mouse o eventos del sistema."""

pass

def update(self, dt):

"""Actualiza la lógica interna del estado basándose en delta time."""

pass

def draw(self, screen):

```
"""Renderiza los elementos visuales correspondientes en la ventana."""  
pass
```

```
class MenuState(State):  
    """Estado que controla el flujo de la pantalla del Menú Principal."""  
    def handle_events(self, events):  
        for event in events:  
            if event.type == pygame.KEYDOWN:  
                if event.key == pygame.K_RETURN: # Presionar ENTER para iniciar  
                    self.manager.change_state("GAME")  
  
    def update(self, dt):  
        pass  
  
    def draw(self, screen):  
        # Color de fondo temático medieval oscuro (Gris muy oscuro azulado)  
        screen.fill((24, 26, 33))  
  
        # Renderizado de texto temporal para validación visual  
        font = pygame.font.SysFont("Arial", 36)  
        title_text = font.render("EL ÚLTIMO BASTIÓN", True, (212, 175, 55)) # Dorado  
        prompt_text = font.render("Presiona ENTER para Jugar", True, (200, 200, 200))  
  
        screen.blit(title_text, (100, 250))  
        screen.blit(prompt_text, (100, 320))  
  
class GameState(State):  
    """Estado que controla la simulación de juego principal."""  
    def handle_events(self, events):  
        for event in events:  
            if event.type == pygame.KEYDOWN:  
                if event.key == pygame.K_ESCAPE: # Presionar ESC para pausar/volver  
                    self.manager.change_state("MENU")  
  
    def update(self, dt):  
        pass  
  
    def draw(self, screen):  
        # Color de fondo que representa el terreno de la fortaleza  
        screen.fill((34, 47, 34))
```

```

font = pygame.font.SysFont("Arial", 36)
game_text = font.render("PANTALLA DE JUEGO (Escenario Principal)", True, (240, 240, 240))
info_text = font.render("Presiona ESC para regresar al Menú", True, (150, 150, 150))

screen.blit(game_text, (100, 250))
screen.blit(info_text, (100, 320))

```

```

class StateManager:
    """
    Máquina de Estados Finitos (FSM) encargada de centralizar la transición
    y delegación de responsabilidades de las diferentes pantallas.
    """

    def __init__(self):
        self.states = {}
        self.current_state = None

    def add_state(self, name, state):
        """Registra un nuevo estado en el gestor."""
        self.states[name] = state

    def change_state(self, name):
        """Realiza la transición segura hacia un nuevo estado."""
        if name in self.states:
            self.current_state = self.states[name]

```

src/core/game_manager.py

```

Python
import pygame
import json
import os
import sys
from src.core.state_manager import StateManager, MenuState, GameState

class GameManager:
    """

```

Clase principal (Engine) que controla el ciclo de vida de la aplicación, la inicialización de Pygame, la carga de configuraciones y el Game Loop.

```
"""
def __init__(self):
    pygame.init()
    self.load_configuration()

    # Configuración de ventana física para PC
    self.width = self.config["screen"]["width"]
    self.height = self.config["screen"]["height"]
    self.screen = pygame.display.set_mode((self.width, self.height))
    pygame.display.set_caption(self.config["screen"]["title"])

    # Sistema de tiempo y rendimiento
    self.clock = pygame.time.Clock()
    self.target_fps = self.config["screen"]["fps"]
    self.is_running = True

    # Instanciación y registro en la FSM
    self.state_manager = StateManager()
    self.state_manager.add_state("MENU", MenuState(self.state_manager))
    self.state_manager.add_state("GAME", GameState(self.state_manager))

    # Definición del estado de arranque
    self.state_manager.change_state("MENU")

def load_configuration(self):
    """Carga la configuración del entorno desde el archivo estático JSON."""
    config_path = os.path.join("config", "settings.json")
    try:
        with open(config_path, "r", encoding="utf-8") as file:
            self.config = json.load(file)
    except (FileNotFoundError, json.JSONDecodeError):
        # Mecanismo de contingencia (Fallback) ante ausencia de archivo
        self.config = {
            "screen": {"width": 1280, "height": 720, "title": "El Ultimo Bastion", "fps": 60}
        }

def run(self):
    """Punto neurálgico que ejecuta el Game Loop principal."""
    while self.is_running:
        # Control de FPS estricto y cálculo de Delta Time (en segundos)
        dt = self.clock.tick(self.target_fps) / 1000.0
```

```

        self.handle_events()
        self.update(dt)
        self.draw()

def handle_events(self):
    """Captura eventos globales del sistema y delega eventos de entrada."""
    events = pygame.event.get()
    for event in events:
        if event.type == pygame.QUIT:
            self.is_running = False

    # Delegación polimórfica al estado activo
    if self.state_manager.current_state:
        self.state_manager.current_state.handle_events(events)

def update(self, dt):
    """Delega la actualización matemática y lógica al estado correspondiente."""
    if self.state_manager.current_state:
        self.state_manager.current_state.update(dt)

def draw(self):
    """Limpia el búfer de pantalla y solicita el dibujado al estado activo."""
    self.screen.fill((0, 0, 0)) # Borrado de pantalla base

    if self.state_manager.current_state:
        self.state_manager.current_state.draw(self.screen)

    pygame.display.flip() # Intercambio de búferes de video (Double Buffering)

def terminate(self):
    """Cierre ordenado de recursos de Pygame y salida del sistema."""
    pygame.quit()
    sys.exit()

```

main.py

```

Python
from src.core.game_manager import GameManager

def main():
    """Punto de entrada oficial e hilo principal del programa."""
    engine = GameManager()
    engine.run()
    engine.terminate()

if __name__ == "__main__":
    main()

```

3. Explicación Técnica

- **Desacoplamiento Estricto mediante FSM (StateManager):** El motor principal (GameManager) no contiene lógica sobre qué pantallas pintar o cómo reaccionar a las teclas específicas del juego. Al encapsular las pantallas en clases hijas independientes de State, podemos escalar e integrar posteriormente los módulos de oleadas y combate de forma aislada sin modificar el archivo troncal del motor.
- **Garantía de Fluidéz vía Delta Time (dt):** La llamada a `self.clock.tick(self.target_fps)` detiene el hilo lo necesario para asegurar un techo de 60 FPS. Al dividir el retorno entre 1000.0, convertimos este valor a segundos. Pasar dt a los métodos de actualización garantiza que, cuando se implemente el movimiento de las entidades en fases futuras, este sea completamente independiente de la tasa de refresco o potencia de la CPU del hardware de pruebas.
- **Gestión Estática Decoupled (settings.json):** Almacenar la resolución y el framerate objetivo en un archivo JSON externo mitiga los problemas de configuración en caliente y permite a los investigadores cambiar los parámetros de estrés gráfico sin recompilar ni alterar el código fuente bajo análisis.
- **Mecanismo de Doble Búfer (pygame.display.flip):** Evita el molesto parpadeo (*tearing*) gráfico calculando todo el renderizado en un búfer secundario oculto en memoria, volcándolo de golpe a la ventana física del monitor en una sola instrucción de hardware.

4. Instrucciones de Ejecución

Para garantizar la reproducibilidad científica y un entorno de ejecución limpio, siga estos pasos desde la consola de comandos de su sistema operativo:

1. **Navegar al directorio raíz:**
 Bash
`cd ruta/hacia/el_ultimo_bastion`
2. **Configurar el entorno virtual de Python (Recomendado):**
 Bash
`python -m venv venv`

3. **Activar el entorno virtual:**

- *En Windows:* venv\Scripts\activate
- *En Linux/macOS:* source venv/bin/activate

4. **Instalar dependencias:**

Bash

pip install -r requirements.txt

5. **Iniciar la ejecución del núcleo:**

Bash

python main.py

Controles de Verificación Visual:

- Al iniciar, se desplegará una ventana de PC de 1280x720 en un estado de menú oscuro.
- Presione la tecla **ENTER** para transicionar fluidamente al estado de simulación (Fondo verde).
- Presione la tecla **ESC** desde la simulación para regresar instantáneamente al menú principal.
- Haga clic en el botón de **Cierre (X)** de la ventana en cualquier momento para abortar la ejecución de manera segura y limpia en la memoria RAM.